FINAL PROJECT REPORT

IoT Smart Watering System

Automatic watering using ESP32, live soil moisture data, LCD monitoring, relay pump control, and Blynk dashboard

Team members	Trần Quang Khả - 104240554 Vũ Xuân Tùng - 104240800
Course / Project	IoT Auto Farming / Smart Watering System
Development platform	ESP32, Arduino IDE, Blynk IoT
Repository	https://github.com/quangkhamap/IOT-Smart-Watering.git
Submission file	One PDF file per team; editable DOCX version prepared for review

Report note: the main report is designed to stay within 10 pages, excluding this title page and the appendix code section.

1. Executive Summary

This report presents the final version of the IoT Smart Watering System. The project uses an ESP32 microcontroller to read live soil moisture data, convert the sensor value into a moisture percentage, display the result on an LCD screen, and control a water pump through a relay module. The system can work in automatic mode based on soil moisture thresholds, and it can also support manual control through the Blynk dashboard.

The final system is functional. It can read soil moisture values, show the moisture percentage and pump status on the LCD, send key values to Blynk, and turn the pump on or off according to the watering logic. The project does not use a traditional dataset or database. Instead, it works with real-time sensor data from the soil moisture sensor.

Item	Final status
Soil moisture reading	Completed - live analog sensor reading from ESP32 GPIO34
LCD display	Completed - shows moisture percentage, raw value, soil status, and pump status
Automatic pump control	Completed - pump turns on when moisture is below the threshold
Manual pump control	Completed - available through Blynk when auto mode is disabled
Blynk monitoring	Completed - sends moisture, pump status, and auto mode status
Repository link	https://github.com/quangkhamap/IOT-Smart-Watering.git

2. Problem Statement and Objectives

Watering small plants manually can be inconvenient because the user has to check soil condition regularly. If the plant is watered too late, the soil becomes too dry. If the plant is watered too often, it can waste water or damage the plant. The project solves this problem by using a simple IoT system that can monitor soil moisture and control watering automatically.

2.1 Main Objectives

- Read real-time soil moisture data from a soil moisture sensor.
- Convert the raw analog sensor value into a clear moisture percentage.
- Display soil moisture, raw value, soil condition, and pump status on an LCD screen.
- Use ESP32 logic to activate a relay and water pump when the soil is dry.

- Support Blynk dashboard monitoring and manual pump control.
- Use simple threshold logic instead of a complex dataset or machine learning model.

2.2 Final Scope

The project focuses on a practical prototype. The main scope is not to predict future plant needs using AI, but to make a reliable embedded IoT system that reacts to current soil moisture values. Therefore, the most important result is the working connection among the sensor, ESP32, relay, pump, LCD display, Wi-Fi, and Blynk dashboard.

3. System Overview

The system is built around the ESP32. The soil moisture sensor provides an analog input value. The ESP32 reads this value through its ADC pin, processes the value, updates the LCD, sends values to Blynk, and controls a relay module. The relay is used because the water pump requires a separate power path and should not be powered directly from an ESP32 GPIO pin.

3.1 Hardware Components

Component	Purpose in the system
ESP32 microcontroller	Main processing board. It reads sensor data, controls the relay, connects to Wi-Fi, and communicates with Blynk.
Soil moisture sensor	Measures soil moisture as an analog value.
Relay module	Works as an electrical switch to control the water pump.
Water pump	Moves water from the container to the plant when watering is needed.
LCD I2C display	Displays moisture percentage, raw sensor value, soil status, and pump status.
Battery holder with switch	Allows the system power to be turned on and off more easily.
Voltage converter / power module	Provides suitable voltage to the ESP32 and connected modules.
Jumper wires and cables	Connect the sensor, relay, display, and power lines.

3.2 Software and Platforms

Software / Library	Use
Arduino IDE	Used to write, upload, and monitor the ESP32 program.
WiFi.h	Connects the ESP32 to Wi-Fi.

BlynkSimpleEsp32.h	Connects the ESP32 to the Blynk IoT dashboard.
Wire.h	Supports I2C communication for the LCD display.
DIYables_LCD_I2C.h	Controls the 16x2 I2C LCD display.

4. System Architecture and Workflow

The workflow is intentionally simple so that the system can be tested and demonstrated clearly.

Soil Moisture Sensor	**ESP32 Processing**	**LCD + Blynk Monitoring**	**Relay Control**	**Water Pump ON/OFF**

4.1 Step-by-Step Operation

1. The soil moisture sensor produces a raw analog value.
2. ESP32 reads the sensor value from GPIO34.
3. The code takes 30 sensor samples and calculates the average to reduce noise.
4. The raw value is converted into a moisture percentage from 0% to 100%.
5. The LCD is updated with the moisture percentage, raw value, soil status, and pump status.
6. The same data is sent to the Blynk dashboard through virtual pins.
7. If automatic mode is enabled and moisture is below the threshold, ESP32 activates the relay and turns on the pump for 30 seconds.
8. If moisture is high enough, the pump remains off.

5. Data Handling

This project does not use a dataset or a local database. The system uses real-time sensor data. Each time the watering task runs, the ESP32 reads the current soil moisture value, processes it immediately, updates the LCD, sends values to Blynk, and decides whether the pump should run.

Data item	Source	Used for
Raw soil value	Analog reading from GPIO34	Calibration and debugging
Moisture percentage	Calculated from raw value	LCD display, Blynk monitoring, pump decision
Soil status	Calculated from moisture percentage	Simple user display: DRY, MID, or WET
Pump status	Relay state controlled by ESP32	LCD and Blynk status display
Auto mode status	Blynk virtual switch V3	Switches between automatic and manual control

Manual pump request	Blynk virtual switch V2	Allows user to turn pump on or off manually

5.1 Sensor Calibration

The soil moisture sensor gives a raw analog number, not a direct percentage. The code uses two calibration values: dryValue = 2450 and wetValue = 1100. These values represent the approximate sensor readings for dry and wet conditions. The map() function converts the raw value into a 0% to 100% moisture percentage. The constrain() function keeps the result inside the valid range.

6. Implementation Details

6.1 Pin Configuration

Pin / Interface	Connected component	Role
GPIO34	Soil moisture sensor signal	Analog input for moisture reading
GPIO26	Relay module input	Digital output to turn pump on/off
GPIO21 SDA	LCD I2C SDA	I2C data line
GPIO22 SCL	LCD I2C SCL	I2C clock line
Wi-Fi	Blynk Cloud	Remote dashboard communication

6.2 Threshold Logic

The system uses threshold-based control. This is easier to understand, easier to debug, and suitable for a small embedded system. In the final code, the pump starts watering when moisture is below 45%. When this condition is met, the relay turns the pump on for 30 seconds. This duration is long enough to increase soil moisture during testing while still preventing the pump from running continuously.

Moisture percentage	Soil status	System behavior
Below 45%	DRY	Pump turns on for 30 seconds in automatic mode
45% to 49%	MID	System monitors the value and keeps checking
50% or above	WET	Pump stays off

6.3 Pump Control

The pump is controlled by a relay. In the code, HIGH means relay ON and LOW means relay OFF. The function setPump() centralizes pump control, updates the pumpRunning variable, and also

sends the pump status to Blynk. In automatic mode, when the soil moisture drops below 45%, the pump runs for a fixed 30-second watering cycle.

6.4 Stability Improvement

To improve the stability of the sensor reading, the code does not depend on a single analogRead() value. It reads 30 samples and returns the average. In automatic mode, the pump is turned off briefly before reading the sensor. This helps reduce electrical noise from the pump and relay, making the moisture reading more reliable.

7. Blynk Dashboard

Blynk is used as the remote monitoring dashboard. The ESP32 connects to Wi-Fi and Blynk using the template information and authentication token. The system sends moisture percentage, pump status, and auto mode status to Blynk. It also receives commands from Blynk for manual pump control and automatic mode selection.

Blynk virtual pin	Direction	Meaning
V0	ESP32 -> Blynk	Soil moisture percentage
V1	ESP32 -> Blynk	Pump status: ON or OFF
V2	Blynk -> ESP32	Manual pump control
V3	Both directions	Automatic mode status and control

7.1 Manual and Automatic Mode

Automatic mode is the main mode. The ESP32 decides when to water based on the moisture threshold. Manual mode is useful for testing and user override. When auto mode is disabled, the user can turn the pump on or off from Blynk. This makes the system more flexible than a fixed automatic-only design.

8. Testing and Results

The project was tested in stages. First, each component was tested separately. Then, all components were connected and tested as a complete system. The final test confirmed that the system can read live soil moisture data, display it on the LCD, send monitoring values to Blynk, and activate the pump for 30 seconds when the moisture level falls below 45%.

Test case	Expected result	Final result
ESP32 Wi-Fi connection	ESP32 connects to Wi-Fi and Blynk	Passed
LCD display startup	LCD shows system messages	Passed
Soil moisture reading	ESP32 reads live values from sensor	Passed

Moisture conversion	Raw value converts to percentage	Passed
Auto pump control	Pump turns on when soil is dry	Passed
Pump stop logic	Pump stays off when moisture is enough	Passed
Manual Blynk control	User can control pump when auto mode is off	Passed
Blynk monitoring	Dashboard receives moisture and status values	Passed

8.1 Final Demonstration Behavior

- When the system starts, the LCD shows that Wi-Fi and Blynk are connecting.
- After connection, the LCD shows that the system is ready.
- Every 5 seconds, the smartWateringTask() function reads the soil moisture value.
- If moisture is below 45%, the pump turns on for 30 seconds, then turns off.
- If moisture is already enough, the pump stays off.
- The LCD and Blynk dashboard update with the latest system status.

9. Challenges and Solutions

Challenge	Impact	Solution
Sensor values can fluctuate	One reading may not represent the real soil condition	Read 30 samples and use the average value
Pump and relay can cause electrical noise	Sensor reading may become unstable during pump operation	Turn off the pump before reading the sensor and add settle delay
Different components require different power behavior	ESP32 GPIO cannot directly drive a pump	Use a relay module and separate pump power path
User needs to understand system state quickly	Serial Monitor is not suitable for normal use	Add LCD display and Blynk dashboard monitoring
Automatic watering still needs user override	Fully automatic control is less flexible during testing	Add manual control through Blynk

10. Conclusion and Future Work

The IoT Smart Watering System successfully demonstrates a complete automatic watering prototype. The ESP32 receives live soil moisture readings, processes the values, displays information on an LCD screen, sends system data to Blynk, and controls the water pump through a relay. The

final result meets the main project objective: reducing manual watering by using a simple and functional IoT control system.

The project is intentionally designed with simple threshold logic instead of a complex AI model. This makes it reliable, easy to explain, and appropriate for a small embedded systems project. The completed features include live sensor reading, LCD monitoring, automatic pump control, manual mode, Wi-Fi connection, and Blynk dashboard integration.

10.1 Possible Future Improvements

- Add a water level sensor for the water tank to prevent the pump from running when the tank is empty.
- Improve the physical control box so the wiring is cleaner and safer.
- Add longer-term data logging if the project needs a history chart or performance analysis.
- Improve calibration for different soil types because different soil can produce different raw sensor values.
- Add pump safety protection such as maximum daily watering time or low-voltage protection.

APPENDIX A - Source Code

The following Arduino code is inserted for reference. The Blynk authentication token is masked in this editable report for safety. Replace YOUR_BLYNK_AUTH_TOKEN with the real token before running the code on the ESP32. The final control rule is: if soil moisture is below 45%, the pump runs for 30 seconds.

```
// ====================================================
// BLYNK CONFIGURATION
// ====================================================
#define BLYNK_TEMPLATE_ID "TMPL6mbQkglXH"
#define BLYNK_TEMPLATE_NAME "Trần Quang Khả"
#define BLYNK_AUTH_TOKEN "YOUR_BLYNK_AUTH_TOKEN"


// ====================================================
// LIBRARIES
// ====================================================
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <Wire.h>
#include <DIYables_LCD_I2C.h>


// ====================================================
// WIFI CONFIGURATION
// ====================================================
char ssid[] = "VGU_Student_Guest";
char pass[] = "";


// ====================================================
// LCD I2C
// ====================================================
DIYables_LCD_I2C lcd(0x27, 16, 2);


// ====================================================
// PINS
// ====================================================
#define SOIL_PIN 34
#define RELAY_PIN 26


// ====================================================
// SENSOR CALIBRATION
// ====================================================
int dryValue = 2450;
int wetValue = 1100;


// ====================================================
// THRESHOLDS
// ====================================================
int pumpOnThreshold = 45;   // below 45% moisture -> pump ON
int pumpOffThreshold = 50;  // 50% or higher -> soil is considered wet enough


// ====================================================
// TIMING
// ====================================================
unsigned long settleDelay = 1500;
unsigned long pumpDuration = 30000;  // pump runs for 30 seconds when soil is dry
unsigned long afterPumpDelay = 1500;
unsigned long taskInterval = 5000;


// ====================================================
// SYSTEM STATE
// ====================================================
bool pumpRunning = false;
bool autoMode = true;
bool manualPumpRequest = false;


BlynkTimer timer;


// ====================================================
```

```
// PUMP CONTROL
// Relay: HIGH = ON, LOW = OFF
// ====================================================
void setPump(bool state) {
  if (state) {
    digitalWrite(RELAY_PIN, HIGH);
    pumpRunning = true;
  } else {
    digitalWrite(RELAY_PIN, LOW);
    pumpRunning = false;
  }


  Blynk.virtualWrite(V1, pumpRunning ? 1 : 0);
}


// ====================================================
// READ SENSOR
// ====================================================
int readSoilAverage() {
  long total = 0;
  int samples = 30;


  for (int i = 0; i < samples; i++) {
    total += analogRead(SOIL_PIN);
    delay(10);
  }


  return total / samples;
}


// ====================================================
// RAW TO MOISTURE
// ====================================================
int getMoisturePercent(int raw) {
  int moisturePercent = map(raw, dryValue, wetValue, 0, 100);
  moisturePercent = constrain(moisturePercent, 0, 100);
  return moisturePercent;
}


// ====================================================
// SOIL STATUS
// ====================================================
String getSoilStatus(int moisturePercent) {
  if (moisturePercent < pumpOnThreshold) {
    return "DRY";
  } else if (moisturePercent >= pumpOffThreshold) {
    return "WET";
  } else {
    return "MID";
  }
}


// ====================================================
// LCD
// ====================================================
void updateLCD(int raw, int moisturePercent, String soilStatus, String actionText) {
  lcd.clear();


  lcd.setCursor(0, 0);
  lcd.print("M:");
  lcd.print(moisturePercent);
  lcd.print("% ");
  lcd.print(soilStatus);


  lcd.setCursor(0, 1);
  lcd.print("R:");
  lcd.print(raw);
  lcd.print(" P:");
  lcd.print(pumpRunning ? "ON " : "OFF");
}


// ====================================================
// BLYNK SEND
```

```
// V0 = Soil Moisture %
// V1 = Pump Status
// V2 = Manual Pump
// V3 = Auto Mode
// ====================================================
void sendStatusToBlynk(int moisturePercent) {
  Blynk.virtualWrite(V0, moisturePercent);
  Blynk.virtualWrite(V1, pumpRunning ? 1 : 0);
  Blynk.virtualWrite(V3, autoMode ? 1 : 0);
}


// ====================================================
// SERIAL
// ====================================================
void printStatus(int raw, int moisturePercent, String soilStatus) {
  Serial.print("Raw: ");
  Serial.print(raw);


  Serial.print(" | Moisture: ");
  Serial.print(moisturePercent);
  Serial.print("%");


  Serial.print(" | Soil: ");
  Serial.print(soilStatus);


  Serial.print(" | Mode: ");
  Serial.print(autoMode ? "AUTO" : "MANUAL");


  Serial.print(" | Pump: ");
  Serial.println(pumpRunning ? "ON" : "OFF");
}


// ====================================================
// BLYNK V2 - MANUAL PUMP
// ====================================================
BLYNK_WRITE(V2) {
  manualPumpRequest = param.asInt();


  if (!autoMode) {
    setPump(manualPumpRequest);
  }
}


// ====================================================
// BLYNK V3 - AUTO MODE
// ====================================================
BLYNK_WRITE(V3) {
  autoMode = param.asInt();


  if (autoMode) {
    setPump(false);
  } else {
    setPump(manualPumpRequest);
  }
}


// ====================================================
// WHEN BLYNK CONNECTED
// ====================================================
BLYNK_CONNECTED() {
  Blynk.virtualWrite(V1, pumpRunning ? 1 : 0);
  Blynk.virtualWrite(V2, manualPumpRequest ? 1 : 0);
  Blynk.virtualWrite(V3, autoMode ? 1 : 0);
}


// ====================================================
// MAIN SMART WATERING TASK
// ====================================================
void smartWateringTask() {
  // MANUAL MODE
  if (!autoMode) {
    int raw = readSoilAverage();
```

```
    int moisturePercent = getMoisturePercent(raw);
    String soilStatus = getSoilStatus(moisturePercent);

    printStatus(raw, moisturePercent, soilStatus);
    updateLCD(raw, moisturePercent, soilStatus, "MAN");
    sendStatusToBlynk(moisturePercent);

    return;
  }

  // AUTO MODE:
  // Tắt bơm trước khi đọc sensor để tránh nhiễu
  setPump(false);
  delay(settleDelay);

  int raw = readSoilAverage();
  int moisturePercent = getMoisturePercent(raw);
  String soilStatus = getSoilStatus(moisturePercent);

  printStatus(raw, moisturePercent, soilStatus);
  updateLCD(raw, moisturePercent, soilStatus, "CHECK");
  sendStatusToBlynk(moisturePercent);

  // If moisture is below 45%, pump water for 30 seconds
  if (moisturePercent < pumpOnThreshold) {
    Serial.println("Action: Moisture below 45% -> Pump ON for 30 seconds");

    setPump(true);
    updateLCD(raw, moisturePercent, soilStatus, "WATER");
    sendStatusToBlynk(moisturePercent);

    delay(pumpDuration);

    setPump(false);
    updateLCD(raw, moisturePercent, soilStatus, "WAIT");
    sendStatusToBlynk(moisturePercent);

    delay(afterPumpDelay);
  } else {
    Serial.println("Action: Moisture enough -> Pump OFF");

    setPump(false);
    updateLCD(raw, moisturePercent, soilStatus, "OK");
    sendStatusToBlynk(moisturePercent);
  }

  Serial.println("---------------------------------------");
}

// ====================================================
// SETUP
// ====================================================
void setup() {
  Serial.begin(115200);
  delay(2000);

  analogReadResolution(12);
  analogSetPinAttenuation(SOIL_PIN, ADC_11db);

  pinMode(RELAY_PIN, OUTPUT);
  setPump(false);

  Wire.begin(21, 22);

  lcd.init();
  lcd.backlight();
```

```
  lcd.setCursor(0, 0);
  lcd.print("Smart Watering");
  lcd.setCursor(0, 1);
  lcd.print("Connecting WiFi");


  Serial.println("Connecting to WiFi and Blynk...");
  Serial.print("SSID: ");
  Serial.println(ssid);


  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);


  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Blynk Connected");
  lcd.setCursor(0, 1);
  lcd.print("System Ready");


  delay(1500);


  timer.setInterval(taskInterval, smartWateringTask);


  Serial.println("======================================");
  Serial.println("Smart Watering + LCD + Blynk");
  Serial.println("WiFi: VGU_Student_Guest");
  Serial.println("AUTO: pump OFF before sensor reading");
  Serial.println("Pump ON when moisture < 45% for 30 seconds");
  Serial.println("Relay logic: HIGH = ON, LOW = OFF");
  Serial.println("======================================");
}


// ====================================================
// LOOP
// ====================================================
void loop() {
  Blynk.run();
  timer.run();
}
```